



# UTM

UNIVERSITI TEKNOLOGI MALAYSIA

---

**FACULTY OF COMPUTING**

**SECP3623-02 DATABASE PROGRAMMING**

**SEMESTER 01 2025/2026**

---

**PROJECT 1**

**GROUP 5**

**LECTURER: DR. SHAMINI A/P RAJA KUMARAN**

| NAME                                | MATRIC NO |
|-------------------------------------|-----------|
| YASMIN BATRISYIA BINTI ZAHIRUDDIN   | A23CS0201 |
| AIN NURNABILA BINTI MOHD AZHAR      | A23CS0207 |
| SYARIFAH DANIA BINTI SYED ABU BAKAR | A23CS0183 |
| NUR FIRZANA BINTI BADRUS HISHAM     | A23CS0156 |

VIDEO LINK: <https://drive.google.com/drive/folders/1-2PYmT5UL24-6DINmTnCfukoIOiATKWq>

## Table of Contents

|                                                |    |
|------------------------------------------------|----|
| 1.0 Introduction                               | 2  |
| 1.1 Project Overview                           | 2  |
| 1.2 Objectives                                 | 2  |
| 1.3 Team Members and Assigned Roles            | 2  |
| 2.0 SQL Implementation Summary                 | 3  |
| 2.1 Database Design (DDL)                      | 3  |
| 2.2 Database Insertion                         | 4  |
| 2.3 Database Modification                      | 5  |
| 3.0 Query Demonstration                        | 6  |
| 3.1 Filtering Queries                          | 6  |
| 3.2 Group by, having                           | 9  |
| 3.3 Truncate functions                         | 9  |
| 3.4 Upper, Concat functions                    | 10 |
| 3.5 Length functions                           | 10 |
| 3.6 Substring, Concat functions                | 11 |
| 3.7 Case when                                  | 11 |
| 3.8 Subqueries - single row                    | 12 |
| 3.9 Subquery - multiple row                    | 12 |
| 3.10 Union operations                          | 13 |
| 3.11 Not exist operations, Subquery correlated | 13 |
| 3.12 Natural Join                              | 14 |
| 3.13 Inner Join                                | 14 |
| 3.14 Left Join                                 | 15 |
| 3.15 Self Join                                 | 15 |
| 4.0 Optimization Section                       | 17 |
| 4.1 B-TREE Indexing                            | 17 |
| 4.2 FULL-TEXT Indexing                         | 18 |
| 5.0 Team Work                                  | 20 |
| 6.0 Conclusion and Future Enhancements         | 21 |

## 1.0 Introduction

### 1.1 Project Overview

The **Movie Ticket Booking System** is chosen as our domain because it represents the high-volume transaction environment suitable for demonstrating database complexity. Modern cinemas require strong backends to handle the bookings, inventory management and financial tracking. This project simulates the backend operations of a cinema chain and manages the flow from movie scheduling to customer payment processing.

### 1.2 Objectives

The primary objectives of this system are:

- **Data Integrity:** To design a schema with strict constraints like foreign keys, cascade delete that prevents scheduling conflicts and orphan records.
- **Operational Efficiency:** To support daily cinema tasks like adding new movies, scheduling showtimes and processing bookings via optimized SQL commands.
- **Analytical Capability:** To provide management with insights into revenue and popularity through complex aggregation and reporting queries.
- **Performance:** To implement indexing strategies that ensure fast retrieval of movie details.

### 1.3 Team Members and Assigned Roles

The following Table 1.3 shows our team composition and specific tasks to each member:

| Team Member                 | Assigned Roles          | Description                                                                                                                                  |
|-----------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Nur Firzana                 | Database Initialization | Set up the schema, define tables and apply constraints.                                                                                      |
| Nur Firzana & Ain Nurnabila | Data Population         | Insert data for each table.                                                                                                                  |
| Ain Nurnabila               | Data Operations         | Demonstrate UPDATE logic for changes and DELETE logic for cancellations.                                                                     |
| Yasmin Batrisyia            | Analysis Queries        | Perform the logic for filtering, grouping and advanced subqueries.                                                                           |
| Syarifah Dania              | Indexing & Optimization | Implements the JOIN logic, creates indexes to optimize query performance, and reruns the whole project to ensure correctness and efficiency. |

Table 1.3

## 2.0 SQL Implementation Summary

### 2.1 Database Design (DDL)

We established the foundation of the system by creating the MovieBooking schema. This involved defining six core tables: Customers, Movies, Showtimes, Seats, Bookings and Payments.

The project begins with the initialization of the database container using command CREATE DATABASE as shown in Figure 2.1.1. This command serves a key purpose which is logical isolation where it creates a dedicated container for all the tables, keeping the project data separate from other system data.

```
# PART A - Database Creation (DDL)

CREATE DATABASE MovieBooking;
USE MovieBooking;
```

Figure 2.1.1

To ensure having strong data integrity, we implemented a comprehensive set of constraints:

- Primary Key: Assigned to every table to uniquely identify records.
- Foreign Key: Defined relationships between tables. Notably, we used ON DELETE CASCADE for the Showtimes table as shown in Figure 2.1.4 to automatically remove schedules if a movie is deleted. ON DELETE SET NULL for Payments table as Figure 2.1.5 to preserve financial history even if a booking is removed.
- Data Validation: Applied CHECK constraints like in Figure 2.1.3 to prevent invalid data entry and UNIQUE constraints on sensitive fields like email as in Figure 2.1.2.
- Defaults: Used DEFAULT CURRENT\_TIMESTAMP for date fields to automate record keeping.

```
CREATE TABLE Customers (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  fName VARCHAR(50) NOT NULL,
  lName VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE, # constraint: UNIQUE
  phone VARCHAR(20),
  registration_date DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Figure 2.1.2

```
CREATE TABLE Movies (
  movie_id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(100) NOT NULL,
  genre VARCHAR(50),
  duration_minutes INT CHECK (duration_minutes > 0),
  release_date DATE,
  rating DECIMAL (3,1),
  description TEXT
);
```

Figure 2.1.3

```
CREATE TABLE Showtimes (
  showtime_id INT AUTO_INCREMENT PRIMARY KEY,
  movie_id INT,
  hall_number INT NOT NULL,
  show_date DATE NOT NULL,
  start_time TIME NOT NULL,
  price DOUBLE NOT NULL,

  FOREIGN KEY (movie_id) REFERENCES Movies(movie_id) ON DELETE CASCADE
);
```

Figure 2.1.4

Beyond standard table creation, the ability to modify the database structure is ALTER TABLE, which simulated a changing business requirement by adding a loyalty\_points column to the Customers table after the initial creation as in Figure 2.1.6. This allows the system to support a rewards program without recreating the entire database. For the DROP TABLE, we implemented a scenario involving a temporary LuckyDraw table shown in Figure 2.1.6. Once the event concluded, used the DROP TABLE command to remove this table, demonstrating resource cleanup.

```
CREATE TABLE Payments (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  booking_id INT,
  payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,
  payment_method VARCHAR(50),
  amount DOUBLE,
  FOREIGN KEY (booking_id) REFERENCES Bookings(booking_id) ON DELETE SET NULL
);
```

*Figure 2.1.5*

```
CREATE TABLE LuckyDraw (
  winner_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_name VARCHAR(50),
  prize_amount DOUBLE
);
ALTER TABLE Customers ADD COLUMN loyalty_point INT DEFAULT 0;
DROP TABLE LuckyDraw;
```

*Figure 2.1.6*

## 2.2 Database Insertion

To simulate a fully operational production environment, we populated the MovieBooking with various dataset. The data insertion process was carefully sequenced to referential integrity constraints defined in the schema.

This phase stresses the populating of a database with initial records to simulate real-world operations of the Movie Ticket Booking System. So, insert operations were performed in all key tables, including Customers, Movies, Showtimes, Seats, Bookings, and Payments. Careful structuring of the data was considered to support the testing of system functionalities, such as the creation of booking, price calculation, loyalty tracking, and validation of payment.

The process of insertion has also ensured that every table would have in excess of 10 records, and this could be a good volume for subsequent demonstrations of UPDATE, DELETE, and queries. Foreign keys were also considered during the insertion process, so first Customers were added, then Movies, Showtimes, and, finally, Bookings and Payments to maintain referential integrity.

Screenshots like Figures 2.2.1–2.2.4 show examples of successful INSERT statements executed in MySQL, along with validation queries like SELECT COUNT(\*) to confirm that the tables were populated correctly. This insertion phase sets the stage for realistic test scenarios and consistency within the database under variable operations.

```

INSERT INTO Customers (fname, lname, email, phone, loyalty_point)
VALUES
('Ali', 'Baba', 'alibaba@gmail.com', '012-3456789', 100),
('Ain', 'Bella', 'ainbella@gmail.com', '013-4567890', 50),
('Yasmin', 'Batrisyia', 'yasmin@gmail.com', '014-5678901', 20),
('Dania', 'Syarifah', 'dania@gmail.com', '016-7890123', 150),
('Firzana', 'Nur', 'firzana@gmail.com', '017-8901234', 200),
('Ahmad', 'Wajdi', 'wajdi@gmail.com', '019-0123456', 50),
('Aik', 'Cheong', 'aikcheong@gmail.com', '010-1234567', 0),
('Mary', 'Chan', 'mary@gmail.com', '012-1122334', 40),
('Thanu', 'Kumaran', 'thanu@gmail.com', '013-2233445', 50),
('Cha', 'Woo', 'chawoomin@gmail.com', '014-5566778', 250);

```

Figure 2.2.1

| customer_id                     | fname   | lname     | email               | phone       | registration_date   | loyalty_point |
|---------------------------------|---------|-----------|---------------------|-------------|---------------------|---------------|
| 1                               | Ali     | Baba      | alibaba@gmail.com   | 012-3456789 | 2025-11-25 08:51:15 | 120           |
| 2                               | Ain     | Bella     | ainbella@gmail.com  | 013-4567890 | 2025-11-25 08:51:15 | 70            |
| 3                               | Yasmin  | Batrisyia | yasmin@gmail.com    | 014-5678901 | 2025-11-25 08:51:15 | 40            |
| 4                               | Dania   | Syarifah  | dania@gmail.com     | 016-7890123 | 2025-11-25 08:51:15 | 170           |
| 5                               | Firzana | Nur       | firzana@gmail.com   | 017-8901234 | 2025-11-25 08:51:15 | 220           |
| 6                               | Ahmad   | Wajdi     | wajdi@gmail.com     | 019-0123456 | 2025-11-25 08:51:15 | 50            |
| 7                               | Aik     | Cheong    | aikcheong@gmail.com | 010-1234567 | 2025-11-25 08:51:15 | 0             |
| 8                               | Mary    | Chan      | mary@gmail.com      | 012-1122334 | 2025-11-25 08:51:15 | 40            |
| 9                               | Thanu   | Kumaran   | thanu@gmail.com     | 013-2233445 | 2025-11-25 08:51:15 | 50            |
| 10                              | Cha     | Woo       | chawoomin@gmail.com | 014-5566778 | 2025-11-25 08:51:15 | 250           |
| * NULL NULL NULL NULL NULL NULL |         |           |                     |             |                     |               |

Figure 2.2.2

```

INSERT INTO Showtimes (movie_id, hall_number, show_date, start_time, price)
VALUES
(1, 1, '2025-12-01', '14:00:00', 20.00),
(1, 2, '2025-12-01', '18:00:00', 22.00),
(2, 1, '2025-12-01', '20:00:00', 25.00),
(3, 3, '2025-12-02', '10:00:00', 15.00),
(4, 2, '2025-12-02', '13:00:00', 18.00),
(5, 4, '2025-12-03', '12:00:00', 24.00),
(5, 4, '2025-12-03', '11:00:00', 24.00),
(6, 1, '2025-12-04', '22:00:00', 16.00),
(7, 5, '2025-12-04', '21:00:00', 20.00),
(10, 3, '2025-12-05', '15:00:00', 23.00);

```

Figure 2.2.3

| showtime_id                     | movie_id | hall_number | show_date  | start_time | price |
|---------------------------------|----------|-------------|------------|------------|-------|
| 1                               | 1        | 1           | 2025-12-01 | 14:00:00   | 23    |
| 2                               | 1        | 2           | 2025-12-01 | 18:00:00   | 25.3  |
| 3                               | 2        | 1           | 2025-12-01 | 20:00:00   | 31.75 |
| 4                               | 3        | 3           | 2025-12-02 | 10:00:00   | 15    |
| 5                               | 4        | 2           | 2025-12-02 | 13:00:00   | 18    |
| 6                               | 5        | 4           | 2025-12-03 | 12:00:00   | 24    |
| 7                               | 5        | 4           | 2025-12-03 | 11:00:00   | 24    |
| 8                               | 6        | 1           | 2025-12-04 | 22:00:00   | 19    |
| 9                               | 7        | 5           | 2025-12-04 | 21:00:00   | 20    |
| 10                              | 10       | 3           | 2025-12-05 | 15:00:00   | 23    |
| * NULL NULL NULL NULL NULL NULL |          |             |            |            |       |

Figure 2.2.4

## 2.3 Database Modification

The modification stage demonstrates how the system handles changes to existing data, which is essential for maintaining accuracy and reflecting real-life cinema operations.

### UPDATE Operations

Multiple UPDATE commands were applied to create simulated business rules inside a cinema system:

- Confirmation of pending bookings: The pending bookings can be updated to "Confirmed" based on the conditional logic of booking ID and status.
- Price changes: The ticket prices have been increased by a fixed percentage on few dates to show the holiday fee.
- Loyalty point bonuses: Extra loyalty points were dedicated to customers having specific criteria.
- VIP Customer Discounts: Using JOIN logic, high-loyalty point customers received booking discounts automatically.

|    |    |   |                     |    |         |
|----|----|---|---------------------|----|---------|
| 19 | 9  | 6 | 2025-11-26 10:00:00 | 24 | Pending |
| 20 | 10 | 8 | 2025-11-26 11:30:00 | 32 | Pending |

Figure 2.3.1

```
UPDATE Bookings
SET status = 'Confirmed'
WHERE booking_id IN (19, 20) AND status = 'Pending';
```

Figure 2.3.2

|    |    |   |                     |    |           |
|----|----|---|---------------------|----|-----------|
| 19 | 9  | 6 | 2025-11-26 10:00:00 | 24 | Confirmed |
| 20 | 10 | 8 | 2025-11-26 11:30:00 | 32 | Confirmed |

Figure 2.3.3

## DELETE Operations

- DELETE statements were used to maintain data cleanliness and remove useless or outdated entries:
- Removing old cancelled bookings to reduce storage and maintain relevance.
- Deleting seats in renovation, by demonstrating deletion with nested queries
- Removing past showtimes based on date comparisons
- Cleaning duplicate seat records using a self-join DELETE
- Removing orphaned payments that do not have an associated booking anymore
- Delete inactive customers without a history of bookings.

Figures 2.3.1–2.3.6 show examples of these UPDATE and DELETE operations, including the result sets, so it is clear how the records have been changed. Such operations demonstrate a system that can keep data as up-to-date and optimized as possible through a stream of modification commands.

```
# Hall 5: 10 seats
(5, 'A1', 'Standard'),
(5, 'A2', 'Standard'),
(5, 'A3', 'Standard'),
(5, 'B1', 'VIP'),
```

Figure 2.3.4

```
DELETE FROM Seats
WHERE seat_id IN (
  SELECT seat_id FROM (
    SELECT seat_id FROM Seats
    WHERE hall_number = 5 AND seat_number IN ('A1', 'A2')
  ) AS temp
);
```

Figure 2.3.5

| seat_id | hall_number | seat_number | seat_type |
|---------|-------------|-------------|-----------|
| 54      | 4           | C3          | Premium   |
| 55      | 4           | C4          | Premium   |
| 58      | 5           | A3          | Standard  |
| 59      | 5           | B1          | VIP       |
| 60      | 5           | B2          | VIP       |

Figure 2.3.6

## 3.0 Query Demonstration

### 3.1 Filtering Queries

Filtering queries demonstrate the system’s ability to retrieve specific information from large datasets based on conditions. These queries form the foundation of reporting, dashboarding, and real-time operations in a cinema management system.

Filtering queries demonstrate the system’s ability to retrieve specific information from large datasets based on conditions. These queries form the foundation of reporting, dashboarding, and real-time operations in a cinema management system.

The filtering section showcases multiple SQL clauses such as:

- WHERE

Retrieves rows that meet specific criteria.

Example: selecting only confirmed bookings.

- AND / OR

Used to combine multiple conditions, enabling more precise filtering.

Example: retrieving bookings that are confirmed and have total\_amount > 50.

- NOT

Filters out unwanted records.

Example: retrieving bookings that are not cancelled.

- BETWEEN

Gets values within a certain numeric or date range.

Example: selecting bookings with total\_amount between 30 and 70.

- LIKE

Used for pattern matching, such as searching movie titles or customer emails.

Example: selecting all movies where the title contains “The”.

- IS NULL / IS NOT NULL

Identifies missing or non-missing values.

Example: finding customers who have no phone number on record.

Figures 3.1.1–3.1.6 in the report display the results of these filtering operations, proving that the system can correctly extract targeted information from various tables. These filtering techniques are essential for generating daily reports, searching user data, and supporting decision-making processes.

```
SELECT movie_id, title, genre, rating, duration_minutes
FROM Movies
WHERE title LIKE '%The%'
ORDER BY rating DESC;
```

Figure 3.1.1

|   | movie_id | title           | genre     | rating | duration_minutes |
|---|----------|-----------------|-----------|--------|------------------|
| ▶ | 3        | The Dark Knight | Action    | 9.0    | 152              |
|   | 7        | The Lion King   | Animation | 7.2    | 118              |
| * | NULL     | NULL            | NULL      | NULL   | NULL             |

Figure 3.1.2



```

SELECT customer_id,
       CONCAT(fName, ' ', lName) AS full_name,
       email,
       phone
FROM Customers
WHERE phone IS NULL;

```

Figure 3.1.3

| customer_id | full_name | email | phone |
|-------------|-----------|-------|-------|
|             |           |       |       |

Figure 3.1.4

```

SELECT s.showtime_id,
       m.title,
       m.genre,
       s.show_date,
       s.start_time,
       s.price,
       s.hall_number
FROM Showtimes s
INNER JOIN Movies m ON s.movie_id = m.movie_id
WHERE s.show_date BETWEEN '2025-12-01' AND '2025-12-31'
AND s.start_time >= '18:00:00'
AND s.price > 20.00
ORDER BY s.show_date, s.start_time;

```

Figure 3.1.5

|   | showtime_id | title             | genre  | show_date  | start_time | price | hall_number |
|---|-------------|-------------------|--------|------------|------------|-------|-------------|
| ▶ | 2           | Avengers: Endgame | Action | 2025-12-01 | 18:00:00   | 25.3  | 2           |
|   | 3           | Inception         | Sci-Fi | 2025-12-01 | 20:00:00   | 31.75 | 1           |

Figure 3.1.6

This query is designed to give a clear and meaningful summary of everything happening in the Bookings table. It starts by using COUNT(\*) to show how many total bookings exist, which is useful for understanding the overall activity in the system. Then, instead of counting each booking status separately with multiple queries, it uses CASE WHEN inside COUNT() to calculate how many bookings are Confirmed, Pending, or Cancelled in one go. This makes the query more efficient and easier to read. For the financial side, functions like SUM() and AVG() help determine how much total revenue the bookings generate and what the average customer spends per booking. To make the numbers look neat and professional, ROUND() formats them to two decimal places, and CONCAT() adds the “RM” currency label so they appear in a proper currency format. Meanwhile, MAX() and MIN() highlight the highest and lowest booking amounts, giving insights into price variations, such as which bookings were the most expensive or the cheapest. Altogether, this query combines counting, conditional logic, number calculations, and formatting to provide a complete, easy-to-understand overview of the booking data.

```

#count + concat + case when + round + sum + avg + max + min
SELECT
COUNT(*) AS total_bookings,
COUNT(CASE WHEN status = 'Confirmed' THEN 1 END) AS confirmed_count,
COUNT(CASE WHEN status = 'Pending' THEN 1 END) AS pending_count,
COUNT(CASE WHEN status = 'Cancelled' THEN 1 END) AS cancelled_count,
CONCAT('RM', ROUND(SUM(total_amount), 2)) AS total_revenue,
CONCAT('RM', ROUND(AVG(total_amount), 2)) AS avg_booking_value,
CONCAT('RM', ROUND(MAX(total_amount), 2)) AS max_booking,
CONCAT('RM', ROUND(MIN(total_amount), 2)) AS min_booking
FROM Bookings;

```

Figure 3.1.7

|   | total_bookings | confirmed_count | pending_count | cancelled_count | total_revenue | avg_booking_value | max_booking | min_booking |
|---|----------------|-----------------|---------------|-----------------|---------------|-------------------|-------------|-------------|
| ▶ | 32             | 29              | 1             | 2               | RM1429.09     | RM44.66           | RM80.00     | RM15.00     |

Figure 3.1.8

## 3.2 Group by, having

In this example, it groups all showtimes by hall\_number. Then, it counts the number of showtimes in each hall and calculates the average ticket price for each hall. Lastly, it filters the grouped result by showing only halls with an average price greater than 15.

```

667 #group by, having
668 • SELECT hall_number,
669         COUNT(*) AS showtime_count,
670         round(AVG(price),2) AS avg_price
671 FROM Showtimes
672 GROUP BY hall_number
673 HAVING AVG(price) > 15;
674

```

| hall_number | showtime_count | avg_price |
|-------------|----------------|-----------|
| 1           | 6              | 24.58     |
| 2           | 4              | 21.65     |
| 3           | 4              | 19        |
| 4           | 4              | 24        |
| 5           | 2              | 20        |

Figure 3.2.1 Group by , having

## 3.3 Truncate functions

Truncate functions are different from drop and delete. Truncate removes all rows from a table, but still keeps the table structure. Meanwhile, drop removes the whole table completely from the database including its structure and removes specific rows from the table using a where condition. In our example, we would like to truncate table payments.

| payment_id | booking_id | payment_date        | payment_method | amount |
|------------|------------|---------------------|----------------|--------|
| 1          | 1          | 2025-11-15 09:31:00 | Credit Card    | 40     |
| 2          | 2          | 2025-11-15 10:01:00 | Debit Card     | 60     |
| 3          | 3          | 2025-11-16 11:31:00 | E-Wallet       | 44     |
| 4          | 4          | 2025-11-16 12:01:00 | Credit Card    | 88     |
| 5          | 5          | 2025-11-17 14:01:00 | Cash           | 75     |
| 6          | 6          | 2025-11-18 08:31:00 | Credit Card    | 45     |
| 7          | 7          | 2025-11-18 09:01:00 | Debit Card     | 36     |
| 8          | 8          | 2025-11-19 13:01:00 | E-Wallet       | 72     |

Figure 3.3.1 Table payments  
before truncate

```
#TRUNCATE
truncate table payments;
```

Figure 3.3.2 SQL Command

| payment_id | booking_id | payment_date | payment_method | amount |
|------------|------------|--------------|----------------|--------|
|------------|------------|--------------|----------------|--------|

Figure 3.3.3 Table payments  
after truncate

### 3.4 Upper, Concat functions

Upper functions will change the whole column value into all in uppercase. Concat functions are to combine the value that is stated. In our example, we would like to retrieve the full name of the customer and make it all in uppercase.

```
653 #UPPER + CONCAT
654 • select fName, lName,
655      concat(fName, ' ', lName) as FullName,
656      upper(concat(fName, ' ', lName)) as UpperFullName
657 from customers;
```

| fName   | lName     | FullName         | UpperFullName    |
|---------|-----------|------------------|------------------|
| Ali     | Baba      | Ali Baba         | ALI BABA         |
| Ain     | Bella     | Ain Bella        | AIN BELLA        |
| Yasmin  | Batrisyia | Yasmin Batrisyia | YASMIN BATRISYIA |
| Dania   | Syarifah  | Dania Syarifah   | DANIA SYARIFAH   |
| Firzana | Nur       | Firzana Nur      | FIRZANA NUR      |
| Ahmad   | Wajdi     | Ahmad Wajdi      | AHMAD WAJDI      |

Figure 3.4.1 Customer full name in uppercase

### 3.5 Length functions

Length will return the number of characters in a string. In our example, we showed the length of the movie title.

```
659 #LENGTH
660 • select title,
661      length(title) as LengthTitle
662 from movies;
```

| title                                | LengthTitle |
|--------------------------------------|-------------|
| Avengers: Endgame                    | 17          |
| Inception                            | 9           |
| The Dark Knight                      | 15          |
| Spider-Man: No Way Home              | 23          |
| Frozen II                            | 9           |
| Joker                                | 5           |
| The Lion King                        | 13          |
| Parasite                             | 8           |
| Titanic                              | 7           |
| Indiana Jones and the Temple of Doom | 17          |

Figure 3.5.1 Length

### 3.6 Substring, Concat functions

Substring (substr) function is to extract part of a string, starting from a specific position for a specific length. Concat function is to combine two or more string values into one string. In our example, we want to retrieve the username of the customer by combining a substring which selects the first name (fName), from position 1 for a two length with the last name (lName).

```
664 #SUBSTR + CONCAT
665 • select fName, lName,
666 substr(fName,1,2),
667 concat(substr(fName,1,2),lName) as userName
668 from customers;
669
```

| fName   | lName     | substr(fName,1,2) | userName    |
|---------|-----------|-------------------|-------------|
| Ali     | Baba      | Al                | AlBaba      |
| Ain     | Bella     | Al                | AlBella     |
| Yasmin  | Batrisyia | Ya                | YaBatrisyia |
| Dania   | Syarifah  | Da                | DaSyarifah  |
| Firzana | Nur       | Fi                | FiNur       |
| Ahmad   | Wajdi     | Ah                | AhWajdi     |
| Alk     | Cheong    | Al                | AlCheong    |

Figure 3.6.1 customer username

### 3.7 Case when

In our example, case when is used to count specific categories based on conditions inside the bookings table. it used inside count() to calculate how many bookings fall into each status. First, for every row where status = 'confirmed', the case returns 1 and other than that return null. so count() only counts the non-null values. Same goes for the pending and cancelled status as well.

```
667 #count + concat + case when + round + sum + avg + max + min
668 • SELECT
669 COUNT(*) AS total_bookings,
670 COUNT(CASE WHEN status = 'Confirmed' THEN 1 END) AS confirmed_count,
671 COUNT(CASE WHEN status = 'Pending' THEN 1 END) AS pending_count,
672 COUNT(CASE WHEN status = 'Cancelled' THEN 1 END) AS cancelled_count,
673 CONCAT('RM', ROUND(SUM(total_amount), 2)) AS total_revenue,
674 CONCAT('RM', ROUND(AVG(total_amount), 2)) AS avg_booking_value,
675 CONCAT('RM', ROUND(MAX(total_amount), 2)) AS max_booking,
676 CONCAT('RM', ROUND(MIN(total_amount), 2)) AS min_booking
677 FROM Bookings;
678
```

| total_bookings | confirmed_count | pending_count | cancelled_count | total_revenue | avg_booking_value | max_booking | min_b |
|----------------|-----------------|---------------|-----------------|---------------|-------------------|-------------|-------|
| 32             | 27              | 3             | 2               | RM1436.19     | RM44.88           | RM80        | RM15  |

Figure 3.7.1 Case when

### 3.8 Subqueries - single row

Subqueries work by evaluating the inner query first and then using its result in the outer query. In this example, this subquery calculates the average loyalty point for all customers. Then, the outer query selects the email of customer whose loyalty points is greater than that average.



Figure 3.8.1 subqueries

### 3.9 Subquery - multiple row

Multiple row subquery returns multiple values, so the outer query must use an operator like IN to compare a single value against many values. In this example, the inner query returns all customer id with bookings of 30 or more. The outer query then selects the first name of the customer whose id appears in that list. This helps us to filter customer based on matching rows in another table.

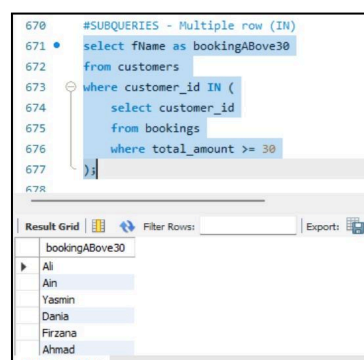
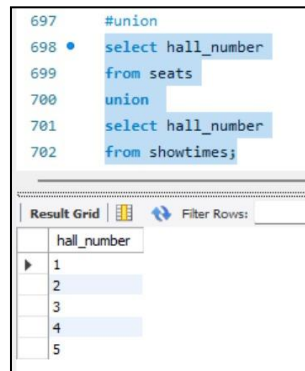


Figure 3.9.1 Subquery multiple row

### 3.10 Union operations

Union is an operation that combines the result of two select statements into one result set. In our example, Union combines the hall\_number results from the seats table and the showtime table into one result.



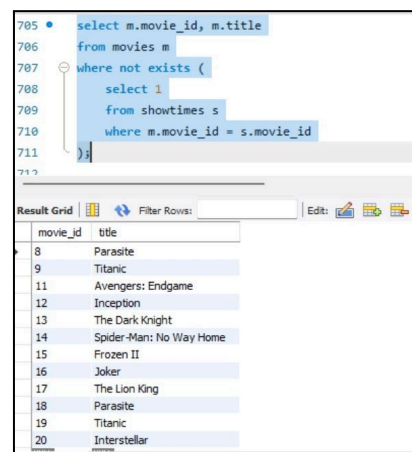
```
697 #union
698 • select hall_number
699   from seats
700   union
701   select hall_number
702   from showtimes;
```

| hall_number |
|-------------|
| 1           |
| 2           |
| 3           |
| 4           |
| 5           |

Figure 3.10.1 Union

### 3.11 Not exist operations, Subquery correlated

Not exist check whether a subquery returns any rows from the outer query where the subquery finds no matching rows. In this example, it lists all movies that do not have any showtime in the showtimes table. For each movie, the subquery checks if a matching movie id exists in showtimes. If none exist, the movie is included.



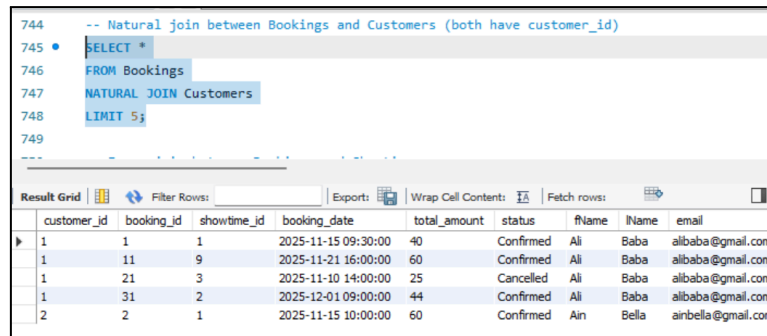
```
705 • select m.movie_id, m.title
706   from movies m
707  where not exists (
708    select 1
709    from showtimes s
710   where m.movie_id = s.movie_id
711  )
```

| movie_id | title                   |
|----------|-------------------------|
| 8        | Parasite                |
| 9        | Titanic                 |
| 11       | Avengers: Endgame       |
| 12       | Inception               |
| 13       | The Dark Knight         |
| 14       | Spider-Man: No Way Home |
| 15       | Frozen II               |
| 16       | Joker                   |
| 17       | The Lion King           |
| 18       | Parasite                |
| 19       | Titanic                 |
| 20       | Interstellar            |

Figure 3.11.1 Not exist and correlated

### 3.12 Natural Join

Natural join automatically finds matching column names, in our example `customer_id`. It joins both tables on that column and shows the combined information.



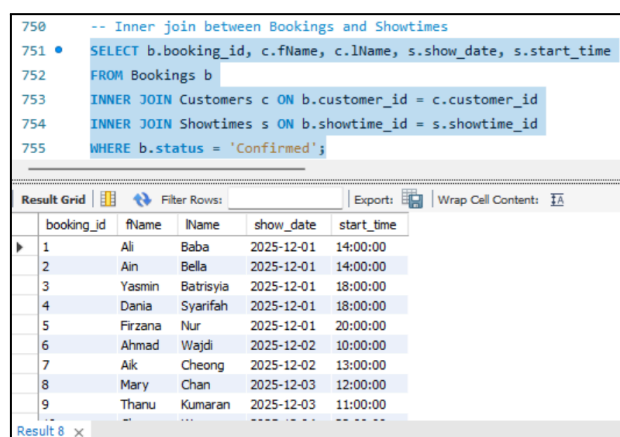
```
744 -- Natural join between Bookings and Customers (both have customer_id)
745 SELECT *
746 FROM Bookings
747 NATURAL JOIN Customers
748 LIMIT 5;
```

|   | customer_id | booking_id | showtime_id | booking_date        | total_amount | status    | fName | lName | email              |
|---|-------------|------------|-------------|---------------------|--------------|-----------|-------|-------|--------------------|
| 1 | 1           | 1          | 1           | 2025-11-15 09:30:00 | 40           | Confirmed | Ali   | Baba  | alibaba@gmail.com  |
| 1 | 11          | 9          | 9           | 2025-11-21 16:00:00 | 60           | Confirmed | Ali   | Baba  | alibaba@gmail.com  |
| 1 | 21          | 3          | 3           | 2025-11-10 14:00:00 | 25           | Cancelled | Ali   | Baba  | alibaba@gmail.com  |
| 1 | 31          | 2          | 2           | 2025-12-01 09:00:00 | 44           | Confirmed | Ali   | Baba  | alibaba@gmail.com  |
| 2 | 2           | 2          | 1           | 2025-11-15 10:00:00 | 60           | Confirmed | Ain   | Bella | ainbella@gmail.com |

Figure 3.12.1 Natural Join

### 3.13 Inner Join

Inner join works by returning only the rows where both tables have matching values in the column used for the join. In this example, we are joining 3 tables; Booking b, Customer c and Showtimes s. First, the inner join checks the customer table and finds rows where `customer_id` matches the one in bookings, ensuring the customer exists. Then, second inner join checks the bookings and finds rows where the `showtime_id` matches the one in showtimes, ensuring that the showtime exists. Lastly, the query filters where the booking status is confirmed. Any booking that does not matched are removed.



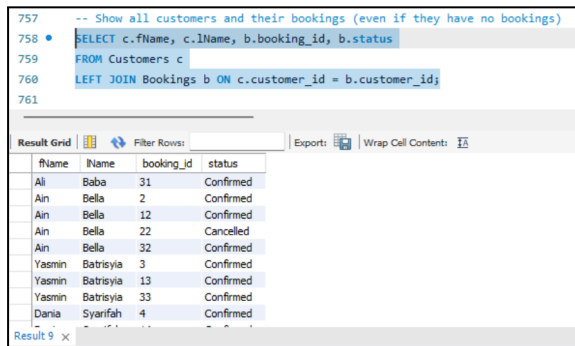
```
750 -- Inner join between Bookings and Showtimes
751 SELECT b.booking_id, c.fName, c.lName, s.show_date, s.start_time
752 FROM Bookings b
753 INNER JOIN Customers c ON b.customer_id = c.customer_id
754 INNER JOIN Showtimes s ON b.showtime_id = s.showtime_id
755 WHERE b.status = 'Confirmed';
```

|   | booking_id | fName   | lName     | show_date  | start_time |
|---|------------|---------|-----------|------------|------------|
| 1 | 1          | Ali     | Baba      | 2025-12-01 | 14:00:00   |
| 2 | 2          | Ain     | Bella     | 2025-12-01 | 14:00:00   |
| 3 | 3          | Yasmin  | Batrisyia | 2025-12-01 | 18:00:00   |
| 4 | 4          | Dania   | Syarifah  | 2025-12-01 | 18:00:00   |
| 5 | 5          | Firzana | Nur       | 2025-12-01 | 20:00:00   |
| 6 | 6          | Ahmad   | Wajdi     | 2025-12-02 | 10:00:00   |
| 7 | 7          | Ali     | Cheong    | 2025-12-02 | 13:00:00   |
| 8 | 8          | Mary    | Chan      | 2025-12-03 | 12:00:00   |
| 9 | 9          | Thanu   | Kumaran   | 2025-12-03 | 11:00:00   |

Figure 3.13.1 Inner Join

### 3.14 Left Join

Left join works by returning all rows from the left table (customers), and it matches them with rows from the right table (Bookings). If a customer does not have any matching record in the booking table, the customer details will still appear but the bookings status will show null shows in figure 3.14.2.



```
757 -- Show all customers and their bookings (even if they have no bookings)
758 SELECT c.fName, c.lName, b.booking_id, b.status
759 FROM Customers c
760 LEFT JOIN Bookings b ON c.customer_id = b.customer_id;
761
```

| fName  | lName     | booking_id | status    |
|--------|-----------|------------|-----------|
| Ali    | Baba      | 31         | Confirmed |
| Ain    | Bella     | 2          | Confirmed |
| Ain    | Bella     | 12         | Confirmed |
| Ain    | Bella     | 22         | Cancelled |
| Ain    | Bella     | 32         | Confirmed |
| Yasmin | Batrisyia | 3          | Confirmed |
| Yasmin | Batrisyia | 13         | Confirmed |
| Yasmin | Batrisyia | 33         | Confirmed |
| Dania  | Syarifah  | 4          | Confirmed |

Figure 3.14.1 Left Join

| fName   | lName   | booking_id | status    |
|---------|---------|------------|-----------|
| Thanu   | Kumaran | 29         | Confirmed |
| Cha     | Woo     | 10         | Confirmed |
| Cha     | Woo     | 20         | Pending   |
| Cha     | Woo     | 30         | Confirmed |
| Charlie | Wong    | NULL       | NULL      |

Figure 3.14.2 Left Join with NULL in booking\_id and status

### 3.15 Self Join

Self join works by joining a table to itself to compare rows inside the same table. In this example, we set bookings a as the first table and bookings b as the second table. Then, if customer\_id from table a matches with table b, means that the bookings are made by the same customer and to prevent duplicate pairs, the second condition comes from retrieving only pairs where a has smaller booking\_id, so only the unique combination are returned. Finally, to add customer information, we do an inner join with table customers to add the customers full name for each booking pair.



```

762 #self join
763 • SELECT
764     a.booking_id AS first_booking,
765     b.booking_id AS second_booking,
766     a.customer_id,
767     CONCAT(c.fName, ' ', c.lName) AS customer_name
768 FROM Bookings a
769     INNER JOIN Bookings b
770         ON a.customer_id = b.customer_id
771         AND a.booking_id < b.booking_id
772     INNER JOIN Customers c
773         ON a.customer_id = c.customer_id;

```

|   | first_booking | second_booking | customer_id | customer_name |
|---|---------------|----------------|-------------|---------------|
| ▶ | 1             | 11             | 1           | Ali Baba      |
|   | 1             | 21             | 1           | Ali Baba      |
|   | 1             | 31             | 1           | Ali Baba      |
|   | 11            | 21             | 1           | Ali Baba      |
|   | 11            | 31             | 1           | Ali Baba      |
|   | 21            | 31             | 1           | Ali Baba      |
|   | 2             | 12             | 2           | Ain Bella     |
|   | 2             | 22             | 2           | Ain Bella     |
|   | 2             | 32             | 2           | Ain Bella     |

Result 10 x

Figure 3.15.1 Self join

### 3.16 Atomic Operations

This transaction proves how SQL keeps important updates safe by treating them as one complete action. This function starts with `START TRANSACTION`, which puts the database into a temporary state while the system tries to apply a 10% discount to eligible VIP customers. The `UPDATE` affects bookings that match certain conditions, for instance, having high loyalty points and a “Not Confirmed” status. Once everything runs well, `COMMIT` saves the changes permanently and the system displays a success message saying the discount was applied. However, if anything goes wrong such as mismatched data or errors, the system uses `ROLLBACK` to undo all changes and protect the database from heavy updates. Due to this, it shows a message saying the update was unsuccessful. This ensures the entire operation is safe, consistent, and either fully applied or not applied at all.

```

START TRANSACTION;
UPDATE Bookings b
INNER JOIN Customers c ON b.customer_id = c.customer_id
SET b.total_amount = ROUND(b.total_amount * 0.90, 2)
WHERE c.loyalty_point >= 150
AND b.status = 'Not Confirmed'
AND b.booking_id <= 15;
-- If all steps ok:
COMMIT;
SELECT 'UPDATE S Completed: 10% VIP discount applied for high-loyalty customers'
-- -- If any step fails:
ROLLBACK;
SELECT 'UPDATE S Unsuccessful' AS Operation_Status;

```

Figure 3.16.1

```

START TRANSACTION ;
UPDATE Bookings b
INNER JOIN Customers c ON b.customer_id = c.customer_id
SET b.total_amount = ROUND(b.total_amount * 0.90, 2)
WHERE c.loyalty_point >= 150
AND b.status = 'Not Confirmed'
AND b.booking_id <= 15;
-- If all steps okay
COMMIT;
SELECT 'UPDATE S Completed: 10% VIP discount applied for high-loyalty customers' AS Operation_Status;
-- If any steps fails
ROLLBACK;
SELECT 'UPDATE S is unsuccessful' AS Operation_Status;

```

Figure 3.16.2

|   |                                                |
|---|------------------------------------------------|
|   | Operation_Status                               |
| ▶ | UPDATE 5 Completed: 10% VIP discount applie... |

Figure 3.16.3

|   |                          |
|---|--------------------------|
|   | Operation_Status         |
| ▶ | UPDATE 5 is unsuccessful |

Figure 3.16.4

## 4.0 Optimization Section

### 4.1 B-TREE Indexing

#### Before Optimization

Before the index was applied, the query that grouped showtimes by hall required MySQL to perform a full table scan, as shown in Figure 4.1.2 where no possible keys or actual keys were used. This meant that all rows had to be read and processed individually, resulting in slower performance and the query took 0.031 seconds to run.

```

EXPLAIN
SELECT hall_number,
       COUNT(*) AS showtime_count,
       AVG(price) AS avg_price,
       ROUND(AVG(price), 2) AS rounded_avg_price,
       MIN(price) AS min_price,
       MAX(price) AS max_price
FROM Showtimes
GROUP BY hall_number
ORDER BY hall_number;

```

Figure 4.1.1 The query used for demonstration of indexing

|   | id | select_type | table     | partitions          | type | possible_keys       | key                 | key_len             | ref                 | rows | filtered | Extra                           |
|---|----|-------------|-----------|---------------------|------|---------------------|---------------------|---------------------|---------------------|------|----------|---------------------------------|
| ▶ | 1  | SIMPLE      | Showtimes | <small>NULL</small> | ALL  | <small>NULL</small> | <small>NULL</small> | <small>NULL</small> | <small>NULL</small> | 10   | 100.00   | Using temporary; Using filesort |

Figure 4.1.2 EXPLAIN output after running the query

15 11:21:24 SELECT hall\_number, COUNT(\*) AS showtime\_count, AVG(price) AS avg\_price, ROUND(AVG(p... 5 row(s) returned

0.031 sec / 0.000 sec

Figure 4.1.3 Duration taken for the query to run

#### After Optimization

After the BTREE index was created on the hall\_number column, the EXPLAIN output showed clear improvements in how MySQL processed the query. The possible\_keys field now displayed idx\_hall\_number, indicating that MySQL could use the index, and the key field confirmed that the

index was actually used during execution. Instead of performing a full table scan, MySQL accessed the grouped data more efficiently through the indexed column. This reduced internal row scanning and improved overall performance. The query also ran faster, taking 0.000 seconds and more smoothly compared to before, demonstrating the impact of proper indexing on optimizing GROUP BY and ORDER BY operations.

```
CREATE INDEX idx_hall_number ON Showtimes(hall_number);
```

Figure 4.1.4 Creation of idx\_hall\_number on hall number in Showtimes table

|   | id | select_type | table     | partitions          | type  | possible_keys   | key             | key_len | ref                 | rows | filtered | Extra               |
|---|----|-------------|-----------|---------------------|-------|-----------------|-----------------|---------|---------------------|------|----------|---------------------|
| ► | 1  | SIMPLE      | Showtimes | <small>NULL</small> | index | idx_hall_number | idx_hall_number | 4       | <small>NULL</small> | 10   | 100.00   | <small>NULL</small> |

Figure 4.1.5 Explain Output after applying B-Tree indexing

25 11:38:42 SELECT hall\_number, COUNT(\*) AS showtime\_count, AVG(price) AS avg\_price, ROUND(AVG(p... 5 row(s) returned

0.000 sec / 0.000 sec

Figure 4.1.6 Time taken for the query to run

## 4.2 FULL-TEXT Indexing

### Before Optimization

To evaluate the impact of FULLTEXT indexing, a comparison was made between searches conducted without and with the index. Initially, the Movies table was searched using the LIKE operator on the title and genre columns to locate the keyword "Action." Without an index, MySQL performed a full table scan, as confirmed by the EXPLAIN output (Figure 4.2.2), resulting in a query execution time of 0.031 seconds.

```
SELECT *
FROM Movies
WHERE title LIKE '%Action%' OR genre LIKE '%Action%'
LIMIT 0, 1000;
```

Figure 4.2.1 Query using LIKE to search for "Action" in title and genre without a FULLTEXT index

|   | id | select_type | table  | partitions          | type | possible_keys       | key                 | key_len             | ref                 | rows | filtered | Extra       |
|---|----|-------------|--------|---------------------|------|---------------------|---------------------|---------------------|---------------------|------|----------|-------------|
| ► | 1  | SIMPLE      | Movies | <small>NULL</small> | ALL  | <small>NULL</small> | <small>NULL</small> | <small>NULL</small> | <small>NULL</small> | 10   | 20.99    | Using where |

Figure 4.2.2 Output table without FULLTEXT Indexing

38 12:07:05 SELECT \* FROM Movies WHERE title LIKE "%Action%" OR genre LIKE "%Action%" LIMIT 0, 1000 3 row(s) returned

0.031 sec / 0.000 sec

Figure 4.2.3 Duration taken for the query to run

### After Optimization

After adding a FULLTEXT index on the title and genre columns, the search query was executed using the MATCH ... AGAINST syntax. This allowed MySQL to directly reference the index to locate only the rows that contained the keyword, rather than scanning every row in the table. The query plan confirmed that the database was using the FULLTEXT index to efficiently retrieve relevant results. Since our dataset is not very large, the query execution time was slightly longer at 0.032 seconds, likely due to index initialization and relevance calculations. Although our dataset is small and the measured execution time was 0.032 seconds, the FULLTEXT index demonstrates its efficiency by reducing the need for full table scans. On larger datasets, we expect a significant performance improvement.

```
ALTER TABLE Movies
ADD FULLTEXT INDEX idx_movie_title_genre (title, genre);
```

Figure 4.2.4

|   | id | select_type | table  | partitions | type     | possible_keys         | key                   | key_len | ref   | rows | filtered | Extra                                       |
|---|----|-------------|--------|------------|----------|-----------------------|-----------------------|---------|-------|------|----------|---------------------------------------------|
| ▶ | 1  | SIMPLE      | Movies | HULL       | fulltext | idx_movie_title_genre | idx_movie_title_genre | 0       | const | 1    | 100.00   | Using where; Ft_hints: sorted, limit = 1000 |

Figure 4.2.5 EXPLAIN output of the search query after adding the FULLTEXT index.

41 12:25:36 SELECT \* FROM Movies WHERE MATCH(title, genre) AGAINST('Action') LIMIT 0, 1000 3 row(s) returned

0.032 sec / 0.000 sec

Figure 4.2.6 Time taken for the query to run after optimization.

## 5.0 Team Work

The following table outlines the team members' reflections on the skills acquired during this project.

| Name          | Reflection                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Yasmin        | After doing my part about DQL, I have gained hands-on understanding of SQL and data retrieval techniques. I know how to filter, sort, aggregate, and join data effectively for a real world application. I also learned the importance of choosing the right query type for the right task or insight so that it can enhance the performance and accuracy in database management. I also learned from my groupmate how the indexing and optimization works in this movie ticket booking system. By applying these queries in my movie ticket booking system makes it possible to generate insights and reports efficiently. |
| Dania         | In this task, my role was to understand how different SQL techniques improve data retrieval. I learned how joins allow multiple tables to be connected so data can be viewed in a meaningful and complete way. I also learned the importance of indexing, which helps the database search faster instead of scanning every row. Through this, I understood how query optimization can reduce processing time and improve overall performance. Overall, this activity helped me see how proper database design and query techniques make systems more efficient, especially when dealing with large amounts of data.         |
| Ain Nurnabila | I could perform insertion, update of data and simple queries with it. I learned what SQL means for the business need when I want to stop writing it like report Revenue Analysis using Aggregate and automatically discount VIP in JOINS. I was able to find my way around some issues with safe mode restrictions and foreign key dependencies, it helped me understand how you can ensure the integrity of your database. Collaborating with my team, I discovered the crucial role that alignment has in guaranteeing data quality upholds everyone's efforts.                                                           |
| Nur Firzana   | As the team member responsible for the Database Foundation, I focused on designing a strong schema and ensuring accurate data population. Creating the database structure with essential constraints like primary key taught me the importance of referential integrity. I also realized that the data insertion                                                                                                                                                                                                                                                                                                            |

|  |                                                                                                                                                                                                                                                                                                                                                                                               |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>must follow a strict sequence, starting with parent tables before dependent ones to avoid any errors. Implementing CHECK and DEFAULT constraints showed how SQL protects data quality, while using ALTER and DROP TABLE demonstrated the need for an adaptable schema. I gained a practical understanding of how a well-structured backend ensures the system's logic and consistency.</p> |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 6.0 Conclusion and Future Enhancements

This project gave us a better grasp of how a complete database system is designed, built and optimized. We learnt how to apply joins, define relationships, organise tables and use indexing to increase performance. We developed confidence in our ability to write queries, analyse results and comprehend how each component of the database supports the system as a whole through practical experience.

Nevertheless, we did encounter a number of difficulties throughout the process of completing this project. These included correcting relationship, mistakes and handling inconsistent data, for instance, some tables initially had inconsistent keys or missing foreign-key relationships, which caused errors during inserts and joins. We also had some trouble in comprehending complicated joins and addressing problems with sluggish searches or duplicate indexes. Coordinating duties and ensuring that everyone used consistent table names, formats and logic were the additional requirements of working in a group.

For future enhancements, we recommend devoting more attention to the early design stage, particularly establishing the ERD and reaching a consensus on standardised table and column names. Errors may be found sooner if testing was done on a regular basis following each significant modification. In addition, using a larger dataset would also allow for a more prominent demonstration of the benefits of FULLTEXT indexing, as performance improvements become more noticeable with increased data volume. The task would also be more organised if questions, choices and problems were recorded as we went. Finally, dedicating more time to testing different query structures and comparing their performance would help us better understand how SQL behaves under different conditions, leading to cleaner and more efficient database operations.

## Marking Rubric

### SQL Implementation (CLO1)

| Criteria                                                      | Excellent (5)                                                                                                         | Good (4)                                                        | Fair (3)                                        | Weak (2)                                           | Poor (1)                         | Marks |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------|----------------------------------------------------|----------------------------------|-------|
| <b>Backend SQL Implementation (DDL/DML/Queries)</b>           | All SQL statements execute flawlessly; covers full range – DDL, DML, SELECT, joins, subqueries, functions.            | Most SQL correct; small syntax or logic issues.                 | Several working statements but limited variety. | Few working statements; repetitive or basic logic. | Non-functional / incomplete SQL. |       |
| <b>Advanced Data Logic (Filtering, Aggregation, Subquery)</b> | Uses complex logic (nested queries, GROUP BY, HAVING, CASE, conditions) effectively.                                  | Correct use of aggregations with minor gaps.                    | Some queries too simple or misapplied.          | Few aggregations or logical errors.                | Not attempted.                   |       |
| <b>Indexing &amp; Optimization</b>                            | Correctly creates BTREE and TEXT indexes; shows analysis before/after; interprets performance gain.                   | Indexes implemented; partial analysis.                          | Indexing attempted without analysis.            | Incomplete index usage.                            | Not attempted.                   |       |
| <b>Transaction Control &amp; Front-End Logic Integration</b>  | Demonstrates atomic operations (START TRANSACTION, COMMIT, ROLLBACK); explains how it fits system workflow (front-end | Transaction logic corrects but limited integration explanation. | Partial transaction implementation.             | Minimal attempt, unclear scenario.                 | Missing transaction logic.       |       |

|                 |                                     |  |  |  |  |  |
|-----------------|-------------------------------------|--|--|--|--|--|
|                 | use case e.g.<br>booking/purchase). |  |  |  |  |  |
| <b>Subtotal</b> |                                     |  |  |  |  |  |

Teamwork and Collaboration (CLO3)

| Criteria                                    | Excellent (5)                                                                                                                              | Good (4)                                              | Fair (3)                                  | Weak (2)                                    | Poor (1)                                           | Marks |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|-------------------------------------------|---------------------------------------------|----------------------------------------------------|-------|
| <b>Team Coordination &amp; Contribution</b> | Clear task division; balanced participation; strong collaboration evident in report and presentation; each member contributes to SQL work. | Good teamwork and participation with minor imbalance. | Some collaboration; limited task clarity. | Minimal teamwork; dominated by few members. | No teamwork evidence; incomplete group submission. |       |
| <b>Subtotal</b>                             |                                                                                                                                            |                                                       |                                           |                                             |                                                    |       |



Documentation & Presentation (CLO4)

| Criteria                                        | Excellent (5)                                                                                                                     | Good (4)                                                 | Fair (3)                                          | Weak (2)                                    | Poor (1)                               | Marks |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|---------------------------------------------------|---------------------------------------------|----------------------------------------|-------|
| <b>Report Documentation</b>                     | Clear and complete report including intro, SQL summary, query screenshots, optimization, transaction explanation, and reflection. | Report complete with minor formatting or clarity issues. | Adequate report but lacks details or screenshots. | Weakly structured or incomplete sections.   | Missing or very poor documentation.    |       |
| <b>Slide Quality &amp; Visual Communication</b> | Slides are professional, organised, visually clear, and consistent with project flow.                                             | Well-prepared slides with minor design issues.           | Slides understandable but inconsistent layout.    | Poorly structured or cluttered slides.      | Missing or unreadable slides.          |       |
| <b>Verbal Presentation</b>                      | Excellent delivery, confident speakers, balanced time usage, fluent explanation, and effective Q&A handling.                      | Good delivery; some coordination issues.                 | Understandable but low energy; moderate Q&A.      | Disorganized delivery or weak explanations. | Unable to present or answer questions. |       |
| <b>Subtotal</b>                                 |                                                                                                                                   |                                                          |                                                   |                                             |                                        |       |